

High-level Digital Design

Lab Assignment 7

High-level design of a signal processing system

0. Introduction and Goals

As mentioned in the previous laboratory assignment, High-Level Synthesis (HLS) allows to generate transfer level (RTL) implementations from high-level specifications, thus providing a framework to explore the design space exchanging latency and resources.

C/C++ descriptions permit to describe hardware at high-level with the support of specific libraries. Using bit-accurate datatypes, it is possible to analyze the impact of the hardware bit sizes on the system performance (area, speed and power consumption).

In this lab, the Algorithmic C (ac) datatypes are used to design, simulate and analyze a signal processing chain as example.

The goals of the lab are:

- To review an existing design of a low-pass filter in C++.
- To adapt it to Algorithmic C bit-accurate types.
- To design and optimize a band-pass FIR filter with a variable-gain amplifier to adjust a constant amplitude of the band-pass signal.
- (OPTIONAL) To study and optimize the trade-off between resources and throughput with the Catapult software.

1. C++ low-pass FIR filter description and simulation

1. The low-pass filter of this example operates at a 10 kHz sampling rate. The band-pass is located at 400 Hz and the stop band (40 dB) at 500 Hz.
2. Copy from Atenea and extract *lab7.zip* on a folder. It will create a folder called *lab7*, with two subfolders, *lab7_1* and *lab7_2*. Enter the *lab7_1* directory.
3. Double-click on the *low_band_pass_filter.xlsx* file. This spreadsheet contains data example for a low-pass and band-pass filter coefficients and the input samples stimuli.
4. The *samples.csv* file has been created from the *low_pass_filter.xlsx* spreadsheet, by saving in CSV format the *samples* sheet. Observe in this sheet:
 - Column *Time* contains the sample time vector. Sample period is 0.1ms (sample rate $f_s = 10$ kHz).
 - Columns *F1* and *F2* compute 400 Hz sine and 500 Hz cosine components, respectively: $\sin(2\pi \cdot 400 \cdot t)$ and $\cos(2\pi \cdot 500 \cdot t)$. Notice that these frequencies as well as the sample period can be set in the *Config* tab of the spreadsheet.
 - Column *F1 + F2* sums the 400 Hz and 500 Hz components to produce the testbench input samples.

5. To simplify the C++ code to access the data, the *samples* sheet has been exported in CSV (text) format to the *samples.csv* file, where column A (*Time*) is used for the time x-axis and column D ($F1 + F2$) is used as the source of the input samples.
6. Similarly, the *coefficients.csv* file has been created saving in CSV format the *Coefficients_lp_400_Hz_10kHz* sheet. Open the *coefficients.csv* file with a text editor and observe that the low-pass filter has 121 coefficients, thus, its order is 121. The filter coefficients are plotted in Fig. 1.

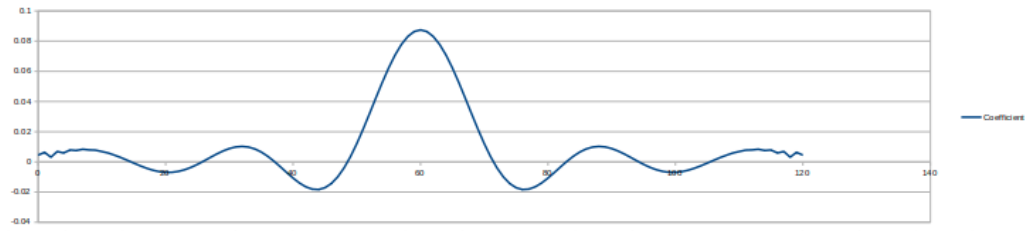


Figure 1. Low-pass filter coefficients.

7. In the */src* directory, open the *fir_filter.h* file. The filter order is defined there, as well as the *fir_filter* function prototype.
8. Open in the same directory the FIR filter design (*fir_filter.cpp*) and check the source code:

```
// High-Level Digital Design (HDD)
//
// Lab evolved from Catapult Online Training (Copyright 2018-2021 Siemens)
//

#include "fir_filter.h"

#pragma hls_design top // To set the Catapult top design block

void fir_filter (const double i_sample,
                 double b[],
                 double &y)
{
    // previous input history, remember across calls
    static double x[TAP_COUNT];
    SHIFT_LOOP:for (int n=TAP_COUNT-1; n>0; n--) {
        x[n] = x[n-1];
    }
    x[0] = i_sample;

    double sum = 0;
    MAC_LOOP:for (unsigned n=0; n<TAP_COUNT; n++) {
        sum += x[n] * b[n];
    }
}
```

```
    }  
    y = sum;  
}
```

The *fir_filter* function is called once per input sample, with the sample being provided on the *i_sample* argument, the array of coefficients provided on the *b* argument, the array of delayed samples on *x* array, *sum* stores the accumulated product of samples and coefficients and the result is returned on the return reference to the *y* argument. Note that *x* is declared as static since prior samples must be available with each new function call. This also allows to initialize the sample history.

9. Analyze in detail the operation of the code loops: *SHIFT_LOOP* and *MAC_LOOP*.
10. In order to perform the simulation, a testbench is needed. The provided *fir_filter_tb.cpp* testbench applies a stream of stimulus values to the design under test and captures the outputs. When the testbench simulation is run, it will read the two csv files, *coefficients.csv* and *samples.csv*, and it will create the *filter_output.csv* file.
11. Open the *fir_filter_tb.cpp* testbench file. The filter's coefficients are stored in the array named *filterCoeffs*; input samples are stored in the vector named *samples* and output samples are stored in the vector named *samples_out*. It is not necessary to analyze in detail the code. Basically, it simulates the filter operation for a set of input samples. The *main* function of the testbench reads the filter coefficients *coefficients.csv* file, and then the loop reads samples from the *samples.csv* file, while saving the output samples into the *samples_out* vector. The *fir_filter* function is called once for each sample read.
12. Since the design and testbench are written in C, they can be compiled in any standard C/C++ compiler and simulated to evaluate the output. In our case, the compiler provided by Catapult is used in order to simplify the next steps. Open a terminal in *lab7_1* directory and run the compiler script:
./compile.sh
13. This script compiles the *fir_filter* design, testbench and CSV utility files, links the object files, and generates the executable for the testbench. Run the executable that has been generated in the same directory:
./fir_filter_tb
14. The executed testbench generates the *filter_output.csv* file.
15. The *filter_output.csv* file can be opened with a spreadsheet to create a graphic of the output samples. Use *Insert -> Chart...* and select *Lines*.
16. The output waveform should look like Fig. 2, which can also be seen in the *o_samples* sheet of the *low_band_pass_filter.xlsx* file. Observe that the 500 Hz signal has been filtered out, while the 400 Hz signal goes through.
17. Obtain the filter output for the combination of 200 + 300 Hz, 300 + 400 Hz, and 500 + 600 Hz input signals.

18. Capture the results and briefly discuss them in the lab report.

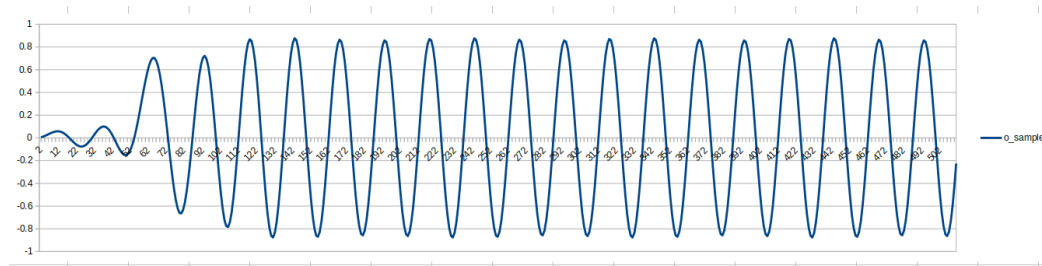


Figure 2. Filtered waveform simulation.

2. Bit-Accurate (AC) Datatypes

As presented in the previous lab, Siemens Catapult HLS tool supports two libraries for specifying arbitrary bit-width signed, unsigned, and fixed-point numbers: The Algorithmic C (AC) library and the System C (SC) library. It is generally recommended to use the AC Datatype library due to the faster C simulation speeds and the support for larger bit-width variables.

The AC Datatype library provides the header files *ac_int.h* and *ac_fixed.h* to support signed/unsigned integer types and fixed point types, respectively.

Table I shows a summary of the syntax for defining these types. Table II summarizes the required bit widths of the results of a few mathematical operations.

Table I. Algorithmic C integer and fixed-point data types.

	Integers	Fixed-Point	Notes
Header File	<i>ac_int.h</i>	<i>ac_fixed.h</i>	+incdir+ in Catapult installation directory
Type definition	<i>ac_int</i> <W,S>	<i>ac_fixed</i> <W,I,S,Q,O>	<i>W</i> : The full bit width of the type <i>S</i> : Boolean flag, true=signed, false=unsigned <i>I</i> : the number of integer bits in W. <i>Q</i> : the rounding method used on each assignment to this type <i>O</i> : the saturation method used on each assignment to this type.

Table II. Bit width growth in mathematical operations.

Operation	Operand 1 bit width	Operand 2 bit width	Result bit width
Add/Subtract	n	m	max(m,n)+1
Multiply	n	m	n+m
Negate	n		n+1
Sum of N operand 1's	n		n+CLOG2(N)

The *double* datatype used for variables in the previous FIR filter description is 64-bits wide, thus requiring a fair amount of hardware for implementation. By replacing the *double* datatype with the *ac_fixed* datatype, hardware resources are saved while improving computation speed.

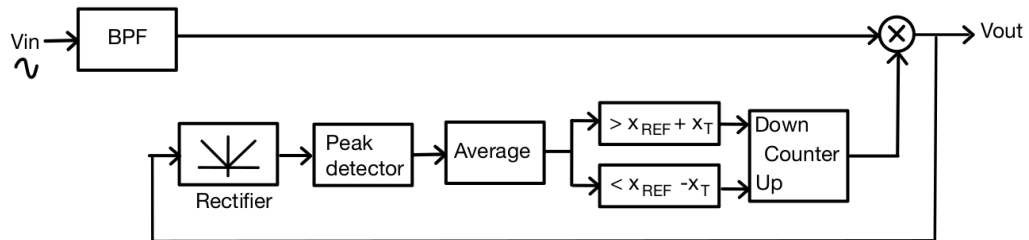
19. Check the header file *lpf_AC.h* in *lab7/lab7_2/src* to see the type definitions and the *lpf_AC.cpp* file for the datatype translation example of the low-pass filter design. Observe also the introduction of a *lpf* function.
20. Run the *compile.csh* script in *lab7/lab7_2* directory:
./compile.sh
21. Run the testbench executable in *lab7/lab7_2* directory:
./lpf_AC_tb

At the end of the run, the testbench generates the *filter_output.csv* file.

22. Open the *filter_output.csv* file and display the filter response. This should correspond to the propagation of the 400 Hz signal and to the rejection of the 500 Hz signal, similar to the previous Fig. 2.
23. Obtain the filter output for the combination of 200 + 300 Hz, 300 + 400 Hz, and 500 + 600 Hz input signals. Does the filter perform correctly its function?
24. Replace the low-pass filter by a band-pass filter. The coefficients are provided in the *low_band_pass_filter.xlsx* spreadsheet. The pass band of this filter is 300 Hz – 400 Hz, while the reject frequencies are < 200 Hz and > 500 Hz.
Important: It is necessary to update the constant that defines the number of coefficients in the header file, otherwise unexpected results can be obtained.
25. Obtain and report the band-pass filter response plot for the four previous combinations of input signals. Does the filter perform correctly its function?

3. Design and optimization of a band-pass filter Automatic Gain Control

Having been introduced to the implementation of FIR filters in bit-accurate Algorithmic C, design a band-pass filter with automatic gain control as shown in the figure:



The low-pass filter corresponds to the previous filter.

The system operation is as follows: The input signal is band-pass filtered, leaving a single harmonic. Then, the output is obtained multiplying this signal by a value from a counter. Let's assume the initial counter value is 1, so the filter signal would appear unaltered at the output.

The output signal is fed back and rectified by taking the absolute value. A peak detector obtains the maximum signal value for some time interval that has to be longer than the period associated with the lowest frequency that the band-pass filter can transmit. After this, the maximum values are averaged with a moving average filter. If this average is higher than a reference level plus a threshold value $X_{REF} + X_T$, this indicates that the output signal is too strong and the counter value is decreased. In case the average is lower than the reference minus a threshold value $X_{REF} - X_T$, the counter has to be incremented. Finally, if the average is in between, the counter value should stay quiet. This way, the feedback loop will compensate the input signal amplitude variations by adapting the multiplication gain (counter value), thus regulating the output signal level to a more or less constant value determined by the X_{REF} value.

Data

- Hardware clock frequency: 100 MHz.
- Sample rate: 10 kHz.
- Filter pass frequencies: 300 Hz - 400 Hz. Ripple: 3 dB.
- Filter stop frequencies: Below 200 Hz and above 500 Hz. Attenuation: -40 dB.
- Input samples format: `ac_fixed<10,2,true>`
- Filter coefficients format: `ac_fixed<12,2,true>`
- Intermediate and output signal format: `ac_fixed<12,4,true,AC_RND,AC_SAT_SYM>`
- Decide the number of moving average samples for the peak detection
- Take the moving average on a reduced number of detected peaks (e.g., 8). You can increase them, if needed, when the system works.
- Set $X_{REF} = X_{MAX}/2$, where X_{MAX} is the maximum amplitude that can be encoded with the used precision.
- Set threshold $X_T \geq 0$. If $X_T = 0$, the counter will produce oscillations. If X_T is too large, the output signal amplitude will lose precision. A few LSBs (Least Significant Bits) should be enough. Tune X_T with the simulations.

- Set a condition to lock the counter if the average level is lower than $x_{REF}/16$. This is to avoid producing high gains when the input signal is too weak or null input.
- Set the gain format to `ac_fixed<12,2,true>`. Initialize the gain value to 1, so the system starts with a unity gain.
- To control the number of averages, a counter should be used. The suggested format is `ac_int<6,false>`.

Procedure

1. Describe the system using Algorithmic C.
2. Simulate the system with 200, 350, 400 and 500 Hz sinusoidal inputs, and different amplitudes (e.g. $x_{0.1}$, $x_{0.5}$, x_{unity} , x_2) to verify the correct gain control.

4. Optional. Design space exploration with Catapult

1. Follow the synthesis procedure of lab6. In case you get a hierarchy error with Catapult, make sure to place the following sentence after the `#include` sentence at the beginning of the top design cpp file:

```
#pragma hls_design top
```
2. Perform a resource – throughput analysis. Obtain different solutions by unfolding totally or partially the loops, and applying pipeline. Propose a solution that minimizes the figure of merit: $area * throughput_cycles$.